

TÜV Prüfstelle Pro — Sprechernotizen

Handout zu praesentation-v8-final-polished.pdf · 12 Hauptfolien · Stand 08.09.2026 ·
Studienprojekt mit produktnaheem Anspruch

VORBEREITUNG AM PRÄSENTATIONSTAG

- Demo-Daten **vor** der Präsentation einspielen (POST /api/admin/demo), nicht live.
- Die Hauptgeschichte hängt nicht an einem laufenden Docker-Stack — die echten Screenshots auf Folie 03 sind der verlässliche Standard.
- Live-Demo nur optional am Ende, wenn Zeit und Setup es erlauben (Ablauf: Anhang B15).
- Das Präsentations-PDF enthält die Sprechernotizen **nicht** — dieses Handout ausdrucken und mitnehmen.
- Zahlen auf Folie 09 tragen „Stand: 08.09.2026“ — nicht als „aktuell“ nennen, ohne die Suite vorher selbst laufen zu lassen.

SPRECHERAUFTEILUNG

- A:** Folie 01–03 · Übergabe an der Leitfrage „Kann man diesem ‚Bestanden‘ trauen?“
- B:** Folie 04–07 (Angriff, Architektur, Datenmodell, Transaktion)
- A:** Folie 08–11 (Rollen, Nachweis, GitHub, Restore)
- B:** Folie 12 (Fazit), danach beide für Fragen

10-MINUTEN-VERSION

Folie 06 (Datenmodell) und 08 (Rollen) überspringen — die Brückensätze stehen bei der jeweiligen Folie. Das ergibt $\approx 13:30$; für echte 10 Minuten zusätzlich Folie 04, 09 und 11 auf die Kernaussage kürzen.

ZEITBUDGET

ZEITBUDGET (Summe der DAUER-Angaben in den Sprechernotizen):

CORE $\sim 15:15$ – alle 12 Folien, ohne "WENN ZEIT IST"

EXTENDED $\sim 24-25$ min – Core plus die "WENN ZEIT IST"-Blöcke

KURZ $\sim 13:30$ – ohne Folie 06 und 08 (dort stehen die Brückensätze).

Für echte 10 Minuten zusätzlich Folie 04, 09 und 11 auf die jeweilige Kernaussage kürzen.

Alles offline: Systemschriften, eingebettete SVG-Icons, eingebettete Screenshots. Keine externen Requests.

LEGENDE

PRESENTER HANDOVER Sprecherwechsel

ÜBERSPRINGBAR in der Kurzfassung weglassen

Q/A mögliche Professorenfrage und knappe Antwort

Rot markiert: Brückensatz beim Überspringen bzw. Formulierungen, die zu vermeiden sind.

Sprecher A

Dauer 0:30

WAS ICH SAGE

"Guten Tag. Wir zeigen Ihnen TÜV Prüfstelle Pro — eine Webanwendung, die den Arbeitsablauf einer TÜV-Prüfstelle digital abbildet: vom Termin über die Prüfung bis zum Prüfergebnis.

Wir haben aber nicht vor, Ihnen jede Maske zu zeigen. Uns interessiert eine einzige Frage, und die zieht sich durch die ganze Präsentation: Wenn eine Fachregel wirklich wichtig ist — wo muss sie dann durchgesetzt werden, damit man sich auf sie verlassen kann?

Einordnung vorweg: Das ist ein Studienprojekt mit produktnahem Anspruch. Es gibt keinen echten Kunden und keine Prüfstelle, die das im Betrieb hat."

ÜBERGANG

"Fangen wir damit an, was da überhaupt gebaut wurde."

WENN ZEIT IST

Die Domäne ist recherchiert, nicht erfunden: § 29 StVZO und die HU-Richtlinie nach Anlage VIII — daher auch die Mangelkategorien OM, GM, EM, GfM.

Arbeitsteilung: grob Frontend/Datenmodell und Backend/Deployment; Details im Projektbericht.

MÖGLICHE FRAGEN

Q Gibt es einen Auftraggeber?

A Nein. Studienprojekt. Die Domäne haben wir aus der Rechtsgrundlage erarbeitet, es gab keine Anforderungsaufnahme bei einer Prüfstelle.

Q Warum React/Express/MariaDB?

A Relationale Daten mit Regeln über Beziehungen (ADR-001, ADR-002) — wir sind bewusst von Firestore weg. Details auf Folie 05 und 06.

VORSICHT

Nicht "Produkt", nicht "Kunde", nicht "produktionsreif" sagen.

VORBEREITUNG AM PRÄSENTATIONSTAG (GILT FÜR DEN GANZEN VORTRAG)

- Demo-Daten VOR der Präsentation einspielen (POST /api/admin/demo), nicht live.
- Die Hauptgeschichte hängt nicht an einem laufenden Docker-Stack: die echten Screenshots auf Folie 03 sind der verlässliche Standard.
- Live-Demo nur optional am Ende, wenn Zeit und Setup es erlauben (Ablauf: B15).
- Das Präsentations-PDF enthält die Sprechernotizen NICHT — diese Notizen als praesentation-v8-final-notizen.pdf ausdrucken und mitnehmen.
- Zahlen auf Folie 09 tragen "Stand: 08.09.2026"; nicht als "aktuell" nennen, ohne die Suite vorher selbst laufen zu lassen.

WAS WIR GEBAUT HABEN

Sprecher A

Dauer 1:20

WAS ICH SAGE

"Links sehen Sie den fachlichen Ablauf: Zu einem Halter gehören Fahrzeuge, zu einem Fahrzeug Termine, zu einem Termin Mängel — und aus den Mängeln ergibt sich das Prüfergebnis.

Rechts ist der Punkt, auf den es ankommt. Ein Termin hat sieben mögliche Status. Sechs davon sind einfach Werte, die jemand setzt. Einer ist es nicht: 'Bestanden'. Ob dieser Wechsel überhaupt erlaubt ist, steht nicht in der Zeile des Termins — es hängt daran, welche Mängel an diesem Termin hängen und ob sie behoben sind.

Das ist der Unterschied zwischen einer Verwaltungsmaske und einer Anwendung mit Fachlogik: Die Gültigkeit eines Zustandswechsels hängt von anderen Datensätzen ab.

Und die Regel dazu steht unten. Sie heißt bei uns WF-01, sie kommt aus § 29 StVZO: Solange ein erheblicher oder ein gefährlicher Mangel offen ist, darf der Termin nicht auf 'Bestanden' wechseln."

ÜBERGANG

"Wie sich das für den Prüfer anfühlt, zeige ich Ihnen an der laufenden Anwendung."

WENN ZEIT IST

Die Kategorien OM / GM / EM / GfM stammen aus der HU-Richtlinie zu § 29 StVZO, Anlage VIII. Der Vorgang wandert außerdem über drei Arbeitsplätze — Empfang, Prüfer, Chef; das ist Folie 08.

MÖGLICHE FRAGEN

Q Ist das ein durchgesetzter Zustandsautomat?

A Nein, und das will ich sauber sagen: Wir erzwingen keinen Übergangsgraphen. Der Status-Endpunkt prüft, dass der Wert einer der sieben bekannten Status ist. Fachlich abgesichert ist genau eine Kante — die auf 'Bestanden'. Das ist bewusst so: das ist die Kante, an der ein falsches Ergebnis entsteht.

Q Warum ist "behoben" nur ein Flag?

A Bewusste Vereinfachung im Projektumfang. Eine echte Nachprüfung mit eigener Historie wäre ein eigenes Thema.

Q Wer legt Termine an?

A Der Empfang. Und zwar immer als "Geplant" — POST /api/termine nimmt gar keinen Status entgegen. Dazu mehr auf Folie 08.

VORSICHT

Nicht sagen, die Anwendung entscheide über die Verkehrssicherheit. Sie entscheidet über das dokumentierte Prüfergebnis.

Keine Marktbehauptungen ("Prüfstellen arbeiten mit Excel") — dazu haben wir keine Erhebung.

Sprecher A → ÜBERGABE AN B AM ENDE DIESER FOLIE

Dauer 1:30

PRESENTER HANDOVER am Ende dieser Folie → Sprecher B

WAS ICH SAGE

"Das sind echte Screenshots aus der laufenden Anwendung, nicht nachgebaut.
Links: ein Termin in Prüfung, noch keine Mängel erfasst. Der Prüfer kann das Ergebnis frei setzen — 'Bestanden' ist wählbar.
In der Mitte: Der Prüfer erfasst einen erheblichen Mangel, hier eine stark korrodierte Bremsleitung. Sofort erscheint der rote Hinweis 'Hauptmangel vorhanden' — nicht bestanden', und die Schaltfläche 'Bestanden' ist deaktiviert.
Das dritte Bild ist bewusst ein anderer Fall — deshalb steht 'Sonderfall' darüber. Hier stand der Termin bereits auf 'Bestanden'. Danach wird ein blockierender Mangel erfasst, und der Server stuft den Termin automatisch auf 'Nicht bestanden' zurück — inklusive der Kennzeichnung 'nicht verkehrssicher'.
Das war kein Klick des Prüfers. Warum diese beiden Schreibrschritte zusammengehören, zeigen wir auf Folie 7.
Und jetzt die Frage, um die es in dieser Präsentation eigentlich geht: Das ist, was der Benutzer sieht. Kann man diesem 'Bestanden' trauen?"

ÜBERGABE AN SPRECHER B

"Genau das haben wir uns auch gefragt — und wir haben versucht, es zu brechen."

WENN ZEIT IST

Die Mängel kommen aus einem StVZO-Katalog mit Kategorie-Badges (OM/GM/EM/GfM); für Fälle ohne Katalogeintrag gibt es einen Freitext-Weg mit Pflicht-Kategorie.

MÖGLICHE FRAGEN

Q Ist das eine Live-Demo?

A Bewusst Screenshots aus einem echten Lauf — die Live-Demo kann ich am Ende gern zeigen, aber die Aussage soll nicht von einem laufenden Docker-Stack im Hörsaal abhängen.

Q Was passiert, wenn der Mangel behoben wird?

A Dann zählt er nicht mehr als Blocker, und 'Bestanden' ist wieder wählbar.
Das ist ein eigener Integrationstest und ein eigener Playwright-Test (Journey-Datei 02-pruefablauf, inklusive Reload-Gegenprobe).

Q Wer darf das Ergebnis überhaupt setzen?

A Nur Prüfer und Chef. Der Empfang sieht diese Schaltflächen gar nicht — und der Server lehnt es zusätzlich ab. Das ist Folie 08.

VORSICHT

- 1) Nur beschreiben, was wirklich zu sehen ist: deaktivierte Schaltfläche und roter Hinweis. Kein "Fehler-Popup", kein "Toast", keine "Warnmeldung" erfinden.
- 2) Bild 1 → 2 ist EIN Ablauf, Bild 3 ist der umgekehrte Fall. Nicht so erzählen, als wären alle drei eine durchgehende Sequenz.

WIR HABEN UNSER EIGENES SYSTEM ANGEGRIFFEN

Sprecher B (übernimmt hier)

Dauer 2:15 — WICHTIGSTE FOLIE DER PRÄSENTATION

WAS ICH SAGE

"Wir haben uns genau diese Frage gestellt — und wir haben nicht darauf vertraut, dass unsere Oberfläche recht hat. Wir haben drei Mal versucht, ein falsches 'Bestanden' in die Datenbank zu bekommen.

Erster Versuch, über die Oberfläche: Die Schaltfläche ist deaktiviert. Das ist gute Bedienung — aber keine Garantie. Der Client läuft auf einem fremden Rechner, den kann man umgehen.

Zweiter Versuch, am Frontend vorbei: Wir rufen die Status-Route direkt mit curl auf. Das ist die echte Antwort, die Sie hier sehen. Der Server lehnt ab und sagt auch warum — mit Verweis auf § 29 StVZO.

Wichtig, und da will ich präzise sein: Das ist eine fachliche Ablehnung, kein HTTP-Fehler. Die Antwort ist HTTP 200 mit ok:false und einer Begründung. Ob das die beste Modellierung ist, dazu sage ich am Ende noch etwas.

Dritter Versuch, an der ganzen Anwendung vorbei: rohes SQL, direkt gegen MariaDB. Kein Frontend, keine API, keine Rechteprüfung dazwischen. Und die Datenbank selbst bricht ab: ERROR 1644, SQLSTATE 45000 — der WF-01-Trigger. Der Status bleibt stehen.

Und jetzt der Teil, der uns beim Aufräumen am meisten gelehrt hat, unten: Dieser dritte Test existierte schon lange. Aber er war fest auf 'docker exec' verdrahtet. In der Pipeline gab es diesen Container nicht — also wurde er dort übersprungen. Ein übersprungener Test sieht in der Zusammenfassung genauso grün aus wie ein bestandener. Die Schutzschicht war da, aber die Pipeline hat sie nicht bewiesen. Die Schwäche lag also nicht im System, sondern im Nachweis. Heute wählt der Test seinen Zugriffsweg zur Laufzeit, und ein eigener CI-Schritt schlägt fehl, wenn er still übersprungen wird."

ÜBERGANG

"Diese drei Ebenen sind kein Zufall — sie sind der Grund, warum die Architektur so aussieht, wie sie aussieht."

WENN ZEIT IST

Der Trigger hängt an BEFORE UPDATE ON termin und schützt den Wechsel auf 'Bestanden'. Er ist kein Beweis, dass 'bestanden trotz Mangel' unter allen Umständen unmöglich wäre: Wird per SQL ein blockierender Mangel zu einem schon bestandenen Termin eingefügt, greift er nicht — diesen Fall löst die API transaktional (Folie 07).

MÖGLICHE FRAGEN

Q Warum antwortet die API mit HTTP-200 und nicht mit 409 oder 422?

A Ehrliche Antwort: Die Anwendung trennt heute die fachliche Ablehnung vom technischen Fehler und gibt dem Frontend einen einheitlichen Vertrag — { ok, reason }. Das ist konsistent und der Client muss keine Statuscodes interpretieren. Aber 409 oder 422 wären eine genauso legitime, in REST sogar üblichere Modellierung. Ich verteidige die 200 nicht als einzig richtige Lösung — das ist ein Punkt, den wir heute anders machen würden. (Ergänzung: Ein 422-Mapper für SQLSTATE 45000 existiert im Fehler-Handler und greift, wenn der Trigger doch auslöst — das ist das Sicherheitsnetz.)

Q Ist Logik in der Datenbank nicht schlechte Praxis?

A Als einziger Ort ja — schwer testbar, schwer auszuliefern. Hier ist sie bewusst redundant zur API: die verständliche Begründung kommt aus Express, der Trigger ist das Netz für alles, was an der Anwendung vorbeischiebt.

Q Warum kein CHECK-Constraint?

A Ein CHECK darf in MariaDB nur Werte derselben Zeile prüfen. WF-01 ist eine Bedingung über eine andere Tabelle — das geht nur mit einem Trigger.

Q Habt ihr auch geprüft, ob die Guards überhaupt etwas tun?

A Ja, und das steht im Quelltext der E2E-Datei: Mit absichtlich ausgebautem Server-Guard blieben die Klick-Journeys grün, weil die Oberfläche die gesperrte Aktion nie abschickt. Deshalb gibt es dort einen Test, der die Anfrage an der Oberfläche vorbeischiekt.

VORSICHT

- 1) NICHT "HTTP-Fehler", "409" oder "422" für die fachliche Ablehnung sagen — es ist HTTP 200 mit ok:false.
- 2) NICHT sagen "damit ist ein falsches Bestanden unmöglich". Sagen: "der Wechsel auf Bestanden ist auf drei Wegen abgesichert".
- 3) Die Nachweis-Lücke nicht kleinreden — sie ist der stärkste Teil der Story.

Sprecher B

Dauer 1:20

WAS ICH SAGE

"Die Architektur ist die Antwort auf die drei Angriffe von eben.

React ist Bedienung und Rückmeldung. Es zeigt, sperrt, blendet aus. Aber es läuft auf einem Rechner, den wir nicht kontrollieren — deshalb darf dort keine Garantie liegen.

Express ist die kontrollierte Grenze. Alles, was dahinter liegt, entsteht unter unserer Kontrolle: Anmeldung, Rechte pro Route, Eingabevalidierung, Fachlogik, Transaktionen. Der Browser spricht nie SQL — er spricht HTTP mit einer API, die entscheidet.

MariaDB sichert das, was unabhängig von jedem Programm gelten muss: Beziehungen über Fremdschlüssel, Eindeutigkeit, Wertebereiche — und für WF-01 zusätzlich der Trigger.

Der Satz in der Mitte ist der Kern: Der Browser darf unterstützen. Der Server muss entscheiden. Die Datenbank muss schützen.

Betrieben wird das On-Premise: ein Server-PC in der Prüfstelle, ein 'docker compose up'. Die API ist im LAN erreichbar, die Datenbank hört nur auf localhost."

ÜBERGANG

"Wenn die Datenbank die letzte Instanz ist, dann muss man sich ansehen, wie die Fachlichkeit dort abgebildet ist."

WENN ZEIT IST

Bewusst kein geschichteter Server (ADR-009): rund 22 Endpunkte; eine Controller/Service/Repository-Aufteilung hätte mehr Indirektion als Nutzen gebracht. Validierung und Auth liegen trotzdem in eigenen Modulen, weil sie ohne Datenbank testbar sein sollen. — Es gibt zusätzlich eine optionale Tauri-Desktop-Hülle um dasselbe gebaute Frontend; ausgeliefert wird Docker Compose.

MÖGLICHE FRAGEN

Q Warum MariaDB und nicht NoSQL?

A Wir hatten zuerst Firestore. Die Daten sind stark relational und die Regeln hängen an Beziehungen — genau das war der Auslöser (ADR-001, ADR-002).

Q Wie weit ist die TypeScript-Migration?

A Bewusst schrittweise (ADR-005). Umgestellt sind die Stellen mit den meisten Verträgen: API-Client, Daten-Hook, Auth-Kontext. Der Rest ist JavaScript; der typecheck läuft trotzdem in jedem CI-Lauf.

Q Wie kommt das Frontend zum Nutzer?

A In der Entwicklung über Vite. Im Betrieb liefert Express das gebaute dist/-Verzeichnis mit aus — ein Origin, kein CORS-Thema.

Q Ist HTTPS im Einsatz?

A Nein, der LAN-Zugriff läuft über HTTP. Das steht als bekannte Grenze in SECURITY.md und ist der erste Punkt, den ich nachziehen würde.

VORSICHT

Nicht "der Server ist sicher" sagen. Sagen: "der Server ist die Stelle, die wir kontrollieren".

Sprecher B

Dauer 1:00

ÜBERSPRINGBAR IN DER 10-MINUTEN-VERSION

BRÜCKENSATZ BEIM ÜBERSPRINGEN

"Das Datenmodell ist klassisch relational in dritter Normalform — zehn Tabellen, und die eine Sache, die Sie sich merken sollten: Welche Mangelkategorie blockiert, steht als Flag in der Datenbank, nicht im Code. Deshalb lesen UI, API und Trigger dieselbe Quelle. Interessanter ist, was passiert, wenn zwei Schreibvorgänge zusammengehören."

WAS ICH SAGE

"Das Datenmodell ist bewusst klassisch relational, dritte Normalform. Die Kette: Halter hat Fahrzeuge, Fahrzeug hat Termine, Termin hat Mängel. Warum relational? Weil genau diese Beziehungen die Fachlichkeit sind. Ein Mangel ohne Termin ergibt keinen Sinn. Und ein Halter darf nicht verschwinden, solange Fahrzeuge auf ihn zugelassen sind — das steht bei uns nicht im Anwendungscode, sondern als Fremdschlüssel mit RESTRICT in der Datenbank. Rechts unten steht der Punkt, der für WF-01 wichtig ist: Die Mangelkategorien sind eine eigene Tabelle, und dort hängt ein Flag 'blockiert_bestanden'. Damit ist die Regel datengetrieben. UI, API und Trigger lesen dieselbe Quelle, und die Zeichenkette 'EM' steht an keiner Stelle hart im Code. Wenn sich die HU-Richtlinie ändert, ändert man eine Zeile Daten — nicht drei Stellen Code."

ÜBERGANG

"Eine Sache kann das Schema allein aber nicht: mehrere Schreibvorgänge zusammenhalten."

WENN ZEIT IST

Zehn Tabellen: vier Kern-Entitäten, vier Stammdaten-Tabellen (status, pruefart, pruefer, mangel_kategorie), benutzer und schema_migration. UUIDs statt Auto-Increment, weil das Frontend optimistische Updates macht und die ID schon beim Anlegen erzeugt. CHECK auf Baujahr (1885–2100) und Kilometerstand (0–3 Mio).

MÖGLICHE FRAGEN

- Q** Welche Normalform, und woran sieht man das?
- A** Dritte, dokumentiert in ADR-002. Halter und Mangel sind eigene Entitäten, genau weil sie vorher eingebettet waren (ADR-008).
- Q** Warum Stammdaten-Tabellen statt ENUM?
- A** Ein ENUM zu ändern heißt ALTER TABLE. Und an der Kategorie hängt ein fachliches Flag — als Fremdschlüsseltabelle ist die Regel pflegbar.
- Q** Wie kommen Schemaänderungen zu einer Bestandsinstallation?
- A** Versionierte Migrationen, jede läuft genau einmal pro Datenbank, protokolliert in schema_migration. Append-only: ausgelieferte Migrationen werden nie editiert. Kein Down-Pfad — der Rückweg ist das Backup (Folie 11).

VORSICHT

Nicht behaupten, die Datenbank verhindere jede fachlich unsinnige Eingabe.
CHECK und FK decken Struktur und Wertebereiche ab; Plausibilität wie
"Hersteller passt zu Modell" liegt in den Validatoren.

Sprecher B

Dauer 1:05

WAS ICH SAGE

"Es gibt einen Fall, den der Trigger nicht abdeckt, und der ist interessant. Ein Termin steht schon auf 'Bestanden'. Jetzt wird ein neuer, unbehobener erheblicher Mangel erfasst. Der Trigger greift hier nicht — er hängt am Status-UPDATE, nicht am Einfügen eines Mangels. Also löst es die Anwendung: Der Endpunkt POST /api/maengel schreibt den Mangel und stuft den Termin im selben Zug auf 'Nicht bestanden' zurück. Beides steckt in einer Transaktion, das sehen Sie links im Quelltext. Warum das zusammengehört: Wenn das INSERT durchginge und das UPDATE fehlschläge, hätten wir einen gespeicherten erheblichen Mangel an einem Termin, der weiter auf 'Bestanden' steht. Genau der Zustand, den die ganze Anwendung verhindern soll — und niemand würde es merken. Entweder beide Änderungen oder keine. Rechts unten ist die echte Antwort der API von heute: terminDemoted true. Das ist exakt das, was Sie auf Folie 3 im Tagesplan gesehen haben — der Termin war grün und wurde rot, ohne dass jemand geklickt hat."

ÜBERGANG

"Bis hier ging es um die Regel. Jetzt zur Frage, wer sie überhaupt auslösen darf."

WENN ZEIT IST

Atomar heißt "alles oder nichts" — nicht "gleichzeitig" und nicht "schnell". Transaktional sind die Server-Vorgänge mit mehreren Schreibschritten: Mangel + Rückstufung, Demo-Seed und Reset. Ein Frontend-Ablauf aus mehreren HTTP-Requests ist ausdrücklich keine Transaktion.

MÖGLICHE FRAGEN

Q Welche Isolationsstufe?

A Der InnoDB-Standard, REPEATABLE READ. Nicht bewusst geändert — für die Größenordnung einer Prüfstelle war das kein Engpass.

Q Was passiert bei einem Verbindungsabbruch?

A Ohne COMMIT rollt MariaDB zurück. Der Code macht zusätzlich im catch-Zweig ein explizites ROLLBACK und gibt die Verbindung im finally frei.

Q Was, wenn zwei Prüfer denselben Termin bearbeiten?

A Beim Status greift WF-01 unabhängig davon. Bei gewöhnlichen Feldern gewinnt heute der letzte Schreibvorgang — wir haben keine Versionsspalte und kein ETag. Das ist eine bewusst offene Stelle (Anhang B9).

Q Ist dieser Fall getestet?

A Auf zwei Ebenen: als Integrationstest gegen echte MariaDB ("Auto-Demotion") und als Playwright-Test durch den Browser.

VORSICHT

Nicht sagen "die ganze Anwendung ist transaktional". Genau benennen, welche Vorgänge in withTransaction laufen.

Sprecher A (übernimmt hier wieder)

Dauer 0:45

ÜBERSPRINGBAR IN DER 10-MINUTEN-VERSION

BRÜCKENSATZ BEIM ÜBERSPRINGEN

"Es gibt drei Rollen — Empfang, Prüfer, Chef —, und der wichtige Satz dazu ist derselbe wie eben: Die Oberfläche blendet aus, die API verweigert. Auch das ist nicht behauptet, sondern getestet — und damit sind wir beim Nachweis."

WAS ICH SAGE

"Drei Rollen. Der Empfang legt Termine und Stammdaten an — mehr nicht. Der Prüfer erfasst zusätzlich Mängel und setzt das Prüfergebnis. Der Chef darf zusätzlich löschen und administrieren.

Wichtiger als die Liste ist die Zeile darunter, und das ist derselbe Gedanke wie bei WF-01: Die Oberfläche blendet aus, was die Rolle nicht darf — das ist Komfort. Durchgesetzt wird es an der API, über eine Middleware, die pro Route ein Recht verlangt: 401 ohne Anmeldung, 403 ohne Recht.

Und auch das ist nicht nur behauptet: Zwei unserer End-to-End-Tests melden sich als Empfang an und schicken den Statuswechsel direkt an die API, ohne Oberfläche. Der Server lehnt ab."

ÜBERGANG

"Womit wir bei der Frage sind, wie wir all diese Behauptungen eigentlich belegen."

WENN ZEIT IST

Passwörter liegen als script-Hash in der Datenbank, Parameter stehen im Hash und sind später erhöhbar. Der Token ist JWT-ARTIG, aber KEIN Standard-JWT: `base64url(payload).base64url(HMAC-SHA256(payload))`, fester Algorithmus, kein Header-Segment — ein `alg=none`-Downgrade ist strukturell ausgeschlossen. 12 Stunden gültig, kein Widerruf.

AUTH_ENABLED ist standardmäßig AN bei `NODE_ENV=production` und AUS in Entwicklung und CI (dort laufen Requests als Dev-Chef, damit die Integrationstests ohne Login funktionieren). In den E2E-Tests ist Auth bewusst eingeschaltet — sonst ließen sich die Rollen-Journeys gar nicht prüfen.

MÖGLICHE FRAGEN

Q Ist Auth im Betrieb wirklich an? © 2020 - 2021, alle Rechte vorbehalten · Sprechernotizen · lokal, nicht veröffentlicht

A Ja — Default an bei `NODE_ENV=production`, und das Docker-Deployment setzt `production`. In Entwicklung und CI ist sie aus, damit die Integrationstests ohne Login laufen; in den E2E-Tests ist sie explizit an.

Q Warum ist `AUTH_ENABLED` in Entwicklung und CI standardmäßig aus?

A Damit lokale Entwicklung und die Server-Integrationstests nicht bei jedem Request ein Login-Setup brauchen — dort laufen Requests als Dev-Chef. Der Default kippt mit `NODE_ENV=production` auf "an", und `.env` kann ihn in beide Richtungen überschreiben. Entscheidend: Die Playwright-E2E-Tests setzen `AUTH_ENABLED=true` ausdrücklich — sonst wären die Rollen- und Login-Journeys sinnlos. Genau dort werden die echten 401- und 403-Antworten geprüft (`03-rollen.spec.js`: "ohne Token liefert die API 401", "Server lehnt einen Statuswechsel des Empfangs auch ohne UI ab" → 403). Auth ist in den Sicherheitstests also NICHT abgeschaltet.

Q Was passiert, wenn ein Prüfer einen Mangel löscht und danach "Bestanden" setzt?

A Dann ist "Bestanden" zulässig — WF-01 blockiert nur, solange ein OFFENER blockierender Mangel existiert. Ist keiner mehr da, gibt es fachlich nichts zu blockieren; im aktuellen Modell ist das der legitime Weg. Der Ablauf "Mangel entfernen → Bestanden wieder möglich → nach Reload noch Bestanden" ist ausdrücklich getestet (`02-pruefablauf.spec.js`, mit `page.reload()`).
Was das System NICHT leistet: einen unveränderlichen Nachweis, WER den Mangel WANN und WARUM gelöscht hat. Das ist der Unterschied zwischen Datenintegrität — die Datenbank ist in sich konsistent — und Nachvollziehbarkeit. Ein böswilliger, berechtigter Prüfer wird durch WF-01 nicht aufgehalten; dagegen hilft nur ein Audit-Log, und das fehlt heute (`SECURITY.md` § 9, Punkt 4 und 7). Wir behaupten das Gegenteil nirgends.

Q Warum kein fertiges JWT, kein bcrypt?

A Fürs On-Premise-Deployment: `node:crypto` ist im Standard enthalten, keine native Build-Toolchain im Container, und das Auth-Modul bleibt ohne Datenbank testbar. Preis: kein Standardformat.

Q Kann man einen Token widerrufen?

A Nein. Zustandslos, läuft nach 12 Stunden ab. Das aktiv-Flag wirkt erst beim nächsten Login. Steht als bekannte Grenze in `SECURITY.md`.

Q Gab es hier mal eine Lücke?

A Ja, und die nenne ich offen: Der generische PATCH auf einen Termin nahm früher `statusCode` entgegen und verlangte nur allgemeines Schreibrecht — der Empfang konnte darüber ein Ergebnis setzen. Geschlossen: `statusCode` wird dort mit HTTP 400 abgelehnt, `POST /api/termine` legt immer "Geplant" an, und es gibt genau eine Status-Route. Abgesichert durch Regressionstests (Anhang B1).

VORSICHT

- 1) Nicht pauschal sagen, jede denkbare Aktion sei rollengeprüft — die Aussage gilt für die API-Routen. Wer direkt in die Datenbank schreibt, umgeht das Rollenmodell; dagegen wirken der Trigger und die Bindung an 127.0.0.1.
- 2) Nicht "JWT" sagen, sondern "JWT-artig, aber kein Standard-JWT".

Sprecher A

Dauer 1:50 — ZWEITE SCHLÜSSELFOLIE

WAS ICH SAGE

"Die Frage ist nicht, wie viele Tests wir haben, sondern was sie beweisen.

Wir haben vier Ebenen, und sie sind bewusst als Leiter angeordnet — von oben nach unten. Je weiter wir nach unten gehen, desto mehr nehmen wir von unserer eigenen Anwendung aus dem Test heraus.

Ebene 1, Unit- und Komponententests: schnell, gut für Grenzfälle. Aber die API ist da teilweise gemockt — die sagen nichts über das Zusammenspiel.

Ebene 2, Server-Integration: die laufende Express-API gegen eine echte MariaDB. Hier wird der WF-01-Guard geprüft, die Rechte-Regression an den Termin-Routen und die automatische Rückstufung.

Ebene 3, End-to-End mit Playwright: echter Browser, echtes Frontend, echte API, echte Datenbank — nichts gemockt. Diese Ebene findet eine Fehlerklasse, die die anderen strukturell nicht finden können: einen Vertragsbruch zwischen Frontend und API. Die Flow-Tests glauben ihrem Mock, die Server-Tests sehen den Browser nicht.

Ebene vier, ganz unten, ist deshalb unser stärkster Einzelbeweis: der direkte SQL-Bypass. Da nehmen wir unsere eigene Anwendung komplett aus dem Weg. Kein Frontend, keine API — nur rohes SQL gegen die

Datenbank. Und wichtig ist mir: Wir zeigen Ihnen hier nicht nur die Fehlermeldung von Folie 4, sondern den Test darunter. Der Fehler beweist, dass der Trigger einmal funktioniert hat. Der Test beweist, dass wir es weiter prüfen — bei jedem Push.

Rechts sehen Sie das Ergebnis der Pipeline zum letzten Commit auf master: 254 Tests grün, 19 Testdateien. Ich habe das am 08.09. lokal reproduziert, es ist dasselbe Ergebnis. Dazu 25 Playwright-Tests in sechs Journey-Dateien, in einem eigenen Job.

Und der rote Kasten ist die Lehre aus der Geschichte von vorhin: Ein übersprungener Test sieht in der Zusammenfassung genauso grün aus wie ein bestandener. Deshalb schlägt ein eigener CI-Schritt fehl, wenn der SQL-Bypass-Test still übersprungen wird."

ÜBERGANG

"Und damit ist GitHub für uns nicht nur Ablage, sondern Teil der Qualitätssicherung geworden."

WENN ZEIT IST

Testentwurfsverfahren: Äquivalenzklassen und Grenzwertanalyse bei den Validatoren, Entscheidungstabellen für die Statuswechsel-Regeln, gezielte Sicherheitsfälle bei Auth (manipulierte Hashes, gefälschte und abgelaufene Tokens, fail-closed bei unbekannter Rolle). Zwei Vitest-Projekte: frontend (jsdom, parallel) und server (node, fileParallelism false — beide Integrationsdateien reseeden dieselbe Datenbank).

MÖGLICHE FRAGEN

Q Sind die 254 aktuell? TÜV Prüfstelle Pro · Sprechernotizen · lokal, nicht veröffentlicht

A Das ist der Stand vom 08.09.2026, aus dem CI-Lauf zum letzten master-Commit — und ich habe die Suite heute lokal gegen den Docker-Stack nachgefahren: 254 grün, 2 übersprungen. Die zwei Übersprungenen sind Platzhalter in den Skip-Zweigen, die nur greifen, wenn kein Stack läuft.

Q Warum antwortet die API bei der abgelehnten Statusänderung mit HTTP 200 und nicht mit 409 oder 422?

A Die Anwendung trennt heute die fachliche Ablehnung vom technischen Fehler und gibt dem Frontend einen einheitlichen Vertrag: { ok, reason }. Der Integrationstest prüft deshalb genau das — Status 200, ok:false, Begründung. Aber 409 oder 422 wären eine legitime und in REST üblichere Modellierung. Ich verteidige die 200 nicht als einzig richtige Lösung; das ist ein Verbesserungspunkt, den wir selbst so benennen.

Q Sind die E2E-Tests wirklich ohne Mocks?

A Ja. Playwright startet API und Vite-Server auf eigenen Ports (8788/5174) gegen eine eigene Datenbank, mit eingeschalteter Authentifizierung. Die Rollen-Journeys sprechen die API zusätzlich direkt an.

Q Wie hoch ist die Coverage?

A Wir weisen bewusst keine Prozentzahl aus — die veröffentlichte stammt von einem Stand vor dem Umbau. Coverage war Nebenprodukt, nicht Ziel.

Q Was fehlt noch?

A CRUD-Integrationstests für die übrigen Endpunkte. Und die schnellen UI-Flow-Tests laufen weiter gegen einen gemockten API-Client — die sind bewusst kein E2E-Ersatz.

VORSICHT

- 1) Zahlen immer mit Stand nennen. Keine Zahl behaupten, die ich nicht aus dem CI-Lauf oder einem eigenen Lauf habe.
- 2) NICHT sagen, die UI-Flow-Tests seien E2E.
- 3) Nicht behaupten, die E2E-Tests deckten den kompletten Funktionsumfang ab — es sind sechs Journey-Dateien mit 25 Tests entlang der wichtigsten Pfade.

A Für PRs gegen master, ja — inklusive der Dependabot-PRs.

VORSICHT

Nicht sagen "die Pipeline garantiert Qualität". Sagen: "sie erzwingt, dass die Aussagen, die wir treffen, bei jedem Commit nachgeprüft werden".

BACKUP / RESTORE

Sprecher A

Dauer 1:20

WAS ICH SAGE

"Wir haben Backups' sagt jeder. Die interessante Frage ist: Wird daraus wieder eine funktionierende Datenbank?

Der Restore-Drill beantwortet das. Er nimmt einen Fingerabdruck der Datenbank — einen normalisierten Dump mit sha256 —, schreibt ein verschlüsseltes Backup, und dann löscht er die Datenbank. Wirklich, mit DROP DATABASE, nicht simuliert. Danach stellt er sie wieder her und vergleicht.

Was Sie links sehen, ist ein echter Lauf von heute. Der Fingerabdruck ist identisch, die Zeilenzahlen stimmen — und dann kommt die Zeile, die mir am wichtigsten ist: 'Trigger greift nach dem Restore'.

Warum das kein Detail ist, steht rechts: Ein Datenbank-Dump ohne Trigger spielt sich völlig fehlerfrei ein. Sie hätten alle Daten zurück, kein Fehlerprotokoll, nichts Auffälliges — und WF-01 wäre weg. Die Regel, um die sich die ganze Präsentation dreht, wäre nach einem Restore stillschweigend verschwunden. Deshalb prüft der Restore, ob der Trigger wieder da ist und ob er auch greift: Ein gesperrtes UPDATE muss weiterhin scheitern.

Und ganz unten die Gegenprobe: Der Drill beschädigt absichtlich ein Backup und verlangt, dass der Restore es ablehnt. Ohne diesen Schritt bliebe der Drill auch dann grün, wenn die Prüfsummen-Kontrolle gar nichts prüft.

Ein Backup ist erst bewiesen, wenn der Restore getestet wurde."

ÜBERGANG

"Damit sind wir zurück bei der Frage vom Anfang."

WENN ZEIT IST

Der Drill läuft zweimal, über zwei Zugriffswege — mit installiertem MariaDB-Client und erzwungen über docker exec, wie auf einem Kunden-Server-PC. Genau dieser zweite Weg hat einen echten Fehler aufgedeckt: Die Verbindungsprobe des Container-Wegs fragte ohne Passwort an und schlug deshalb immer fehl. Auf jeder Entwicklermaschine blieb das unsichtbar, weil dort der andere Weg griff. Auf einem Kunden-PC ohne Client hätte es bedeutet: kein Backup, kein Restore.

MÖGLICHE FRAGEN

Q Läuft der Drill gegen die echte Datenbank?

218 / 8 Schritte 05.08.2020 · Notizen · lokal, nicht veröffentlicht

A Nein, er zerstört sie ja. Das Skript weigert sich zu starten, wenn der Datenbankname nicht mit tuv_drill beginnt — sonst könnte ein Tippfehler die Produktionsdatenbank löschen.

Q Woher kommt das Schema im Drill?

A Aus der Anwendung selbst: Migrationen, Seeds und Trigger entstehen genau so wie beim Kunden. Ein Drill gegen ein handgeschriebenes Schema würde einen Fehler in den Migrationen nicht aufdecken.

Q Wird dem Exit-Code vertraut?

A Nein. Die Pipeline verlangt, dass die entscheidenden Zeilen im Protokoll stehen: "RESTORE-DRILL BESTANDEN", "Beschaedigtes Backup korrekt abgelehnt" und "Trigger greift nach dem Restore".

Q Ist das Backup verschlüsselt?

A AES-256 mit PBKDF2. Ohne gesetzten Schlüssel bricht das Skript ab, statt einen Klartext-Dump mit Halterdaten zu schreiben. Und der Drill läuft bewusst über den verschlüsselten Pfad.

Q Was ist noch offen?

A Die Zeitsteuerung am Server-PC — das gehört ins Kunden-Setup, nicht ins Repository. Der Offsite-Sync für Tier 3. Die Alarmierung bei Fehlschlag am Standort. Und Point-in-Time-Recovery über die Binlogs ist konfiguriert, aber Handarbeit und nicht automatisiert geprüft.

VORSICHT

- 1) NICHT sagen "das Backup ist vollständig automatisiert". Tier 2 und der Restore-Weg sind belegt; Zeitsteuerung, Tier 3 und Alarmierung sind offen.
- 2) NICHT sagen, Point-in-Time-Recovery sei getestet.

FAZIT

Sprecher B (schließt ab; A und B stehen gemeinsam für Fragen bereit)

Dauer 1:00

WAS ICH SAGE

"Damit zurück zu der Frage von Folie 3: Kann man diesem 'Bestanden' trauen?

Ja — und zwar nicht, weil wir gut aufgepasst haben, sondern weil es nicht davon abhängt, dass ein Button deaktiviert bleibt.

Drei Dinge fassen die Arbeit zusammen. Durchsetzen: an drei Stellen, weil jede Stelle einen anderen Weg abdeckt. Testen: bis hinunter zum rohen SQL, an der eigenen Anwendung vorbei. Wiederherstellen: bewiesen, nicht behauptet — die Datenbank wird im Drill wirklich gelöscht.

Und weil wir gerade bei Ehrlichkeit sind, eine Sache, die wir heute anders machen würden. Wenn der Server ein unzulässiges 'Bestanden' ablehnt, antwortet er mit HTTP 200 und einem `ok:false` im Körper. Das ist konsistent für unser Frontend, aber es macht die Ablehnung auf Protokollebene unsichtbar. Ein 409 oder 422 wäre die klarere Modellierung. Das ist kein Fehler, der etwas kaputt macht — aber es ist der Punkt, an dem ich am ehesten nachbessern würde.

Vielen Dank für Ihre Aufmerksamkeit — wir freuen uns auf Ihre Fragen."

ÜBERGANG

— Ende. Danach Fragen; Anhang B1–B15 steht als Beleg bereit.

WENN ZEIT IST

Die Einordnung bleibt: Studienprojekt mit produktnahe Anspruch. Offen sind TLS im LAN, ein Änderungsprotokoll und das Offsite-Backup — alles dokumentiert in SECURITY.md § 9 und im Anhang B10.

MÖGLICHE FRAGEN

Q Was würden Sie als Erstes nachziehen?

A TLS im LAN, danach ein Änderungsprotokoll — `req.benutzer` liegt in jedem Request bereit, das ist wenig Aufwand für viel Nachvollziehbarkeit.

Q Ist das produktionsreif?

A Nein, und das behaupten wir nicht. Produktnahe Architektur und ein produktnaher Auslieferungsweg — für echten Betrieb fehlen mindestens TLS, ein Audit-Log und die Zeitsteuerung der Backups am Standort.

Q Was war die wichtigste Erkenntnis?

A Dass die Frage "wo wird eine Regel durchgesetzt" wichtiger ist als "ist die Regel implementiert". Und zweitens: Eine Zusicherung ohne automatisierten Beleg altert sofort — das war die Lehre aus dem übersprungenen Test.

VORSICHT

Keine Roadmap aufsagen, keine Checkliste. Eine Reflexion, dann Schluss.

Nicht in Produktversprechen rutschen.